

Laboratory work 15

Deploying Your Application

This chapter will explain how to deploy your backend and frontend to a server. Successful deployment is a key part of the software development process, and it is important to learn how a modern deployment process works. There are a variety of cloud servers or **PaaS** (short for **Platform-as-a-Service**) providers available, such as **Amazon Web Services (AWS)**, DigitalOcean, Microsoft Azure, Railway, and Heroku.

In this book, we are using AWS and Netlify, which support multiple programming languages that are used in web development. We will also show you how to use Docker containers in deployments.

In this chapter, we will cover the following topics:

- Deploying the backend with AWS
- Deploying the frontend with Netlify
- Using Docker containers

Technical requirements

The Spring Boot application that we created in *Chapter 5, Securing Your Backend*, is required (<https://github.com/PacktPublishing/Full-Stack-Development-with-Spring-Boot-3andReact-Fourth-Edition/tree/main/Chapter05>), as is the React app that we used in *Chapter*

16, Securing Your Application

(<https://github.com/PacktPublishing/Full-StackDevelopmentwith-Spring-Boot-3-and-React-Fourth-Edition/tree/main/Chapter16>).

A Docker installation is necessary for the final section of the chapter.

Deploying the backend with AWS

If you are going to use your own server, the easiest way to deploy a Spring Boot application is to use an executable **Java ARchive (JAR)** file. With Gradle, an executable JAR file can be

created using the Spring Boot Gradle wrapper. You can build your project using the following Gradle wrapper command in your project folder:

```
./gradlew build
```

Alternatively, you can run a Gradle task in Eclipse by right-clicking **Project** in the Project Explorer, navigating to **Window | Show View | Other**, and selecting **Gradle | Gradle Tasks** from the list. This opens a list of Gradle tasks, and you can start the build process by double-clicking the **build** task, as illustrated in the following screenshot. If the Gradle tasks window is empty, click the root folder of the project in Eclipse:

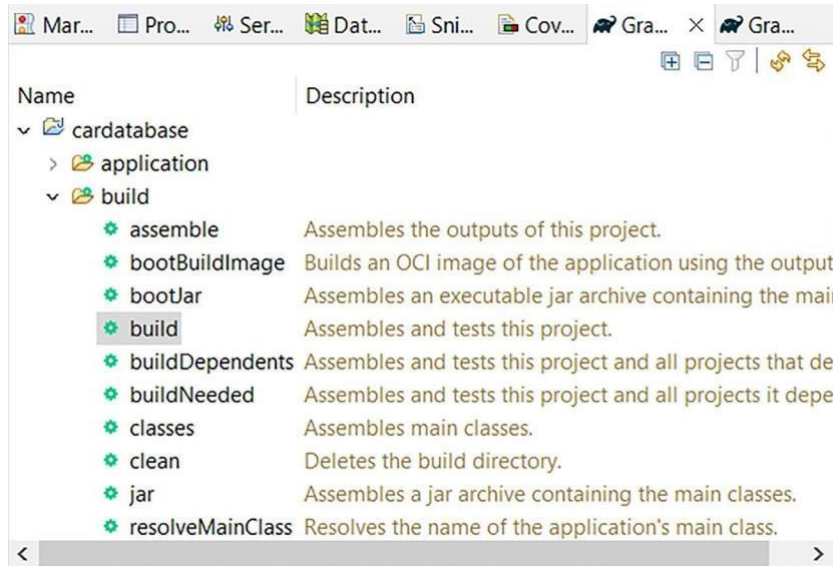


Figure 17.1: Gradle tasks

This creates a new `build/libs` folder to your project, where you will find JAR files. By default, two JAR files are created:

- The file with extension `.plain.jar` contains Java bytecode and other resources, but it doesn't contain any application framework or dependencies.
- The other `.jar` file is a fully executable archive that you can run using the `java -jar your_appfile.jar` Java command, as illustrated in the following screenshot:

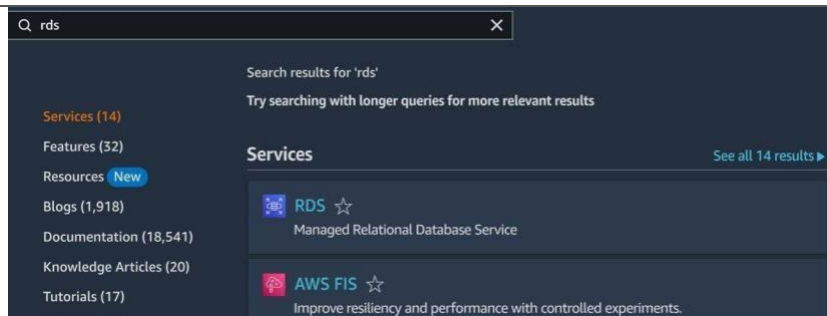


Figure 17.3: RDS

- Click the **Create database** button to begin the database creation process:



Figure 17.4: Create database 3.

Select **MariaDB** from the database engine options:

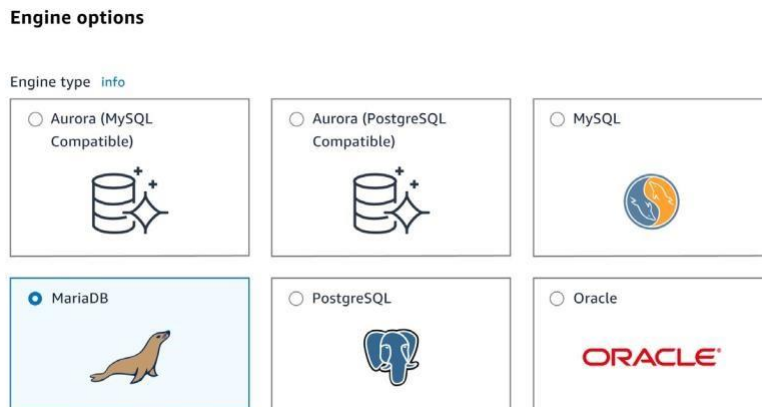


Figure 17.5: Engine options

- Select **Free tier** from the templates.
- Type a name for your database instance and the password for the database master user. You can use the default username (*admin*):

Settings

DB instance identifier [Info](#)

Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.

cardatabase

The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 60 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen

▼ Credentials Settings

Master username [Info](#)

Type a login ID for the master user of your DB instance.

admin

1 to 16 alphanumeric characters. The first character must be a letter.

☐ Manage master credentials in AWS Secrets Manager

Manage master user credentials in Secrets Manager. RDS can generate a password for you and manage it throughout its lifecycle.

 If you manage the master user credentials in Secrets Manager, some RDS features aren't supported.
[Learn more](#) 

☐ Auto generate a password

Amazon RDS can generate a password for you, or you can specify your own password.

Master password [Info](#)

.....

Figure 17.6: Database instance name

6. Select **Yes** under the **Public access** section to allow public access to your database:

Public access [Info](#)

☒ Yes

RDS assigns a public IP address to the database. Amazon EC2 instances and other resources outside of the VPC can connect to your database. Resources inside the VPC can also connect to the database. Choose one or more VPC security groups that specify which resources can connect to the database.

☐ No

RDS doesn't assign a public IP address to the database. Only Amazon EC2 instances and other resources inside the VPC can connect to your database. Choose one or more VPC security groups that specify which resources can connect to the database.

Figure 17.7: Public access

7. In the **Additional configuration** section at the bottom of the page, name your database cardb:

▼ Additional configuration

Database options, encryption turned on, backup turned on, backtrack turned off, maintenance, CloudWatch Logs, delete protection turned off.

Database options

Initial database name [Info](#)

If you do not specify a database name, Amazon RDS does not create a database.

DB parameter group [Info](#)

Option group [Info](#)

Figure 17.8: Additional configuration



Note! The database will not be created if the name is left empty.

8. Finally, click the **Create database** button. RDS will start to create your database instance, which might take a few minutes.
9. After your database is successfully created, you can press the **View connection details** button to open a window that shows the connection details to your database. The **Endpoint** is the address of your database. Copy the connection details for later use:

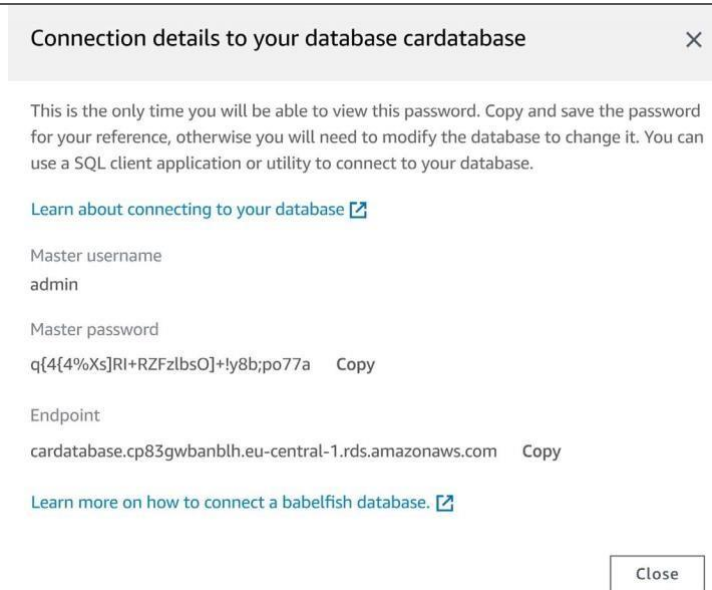


Figure 17.9: Connection details

10. Now, we are ready to test our database. In this phase, we will use our local Spring Boot application. For this, we have to allow access to our database from outside. To change this, click on your database in the RDS database list. Then, click **VPC security groups**, as shown in the next screenshot:

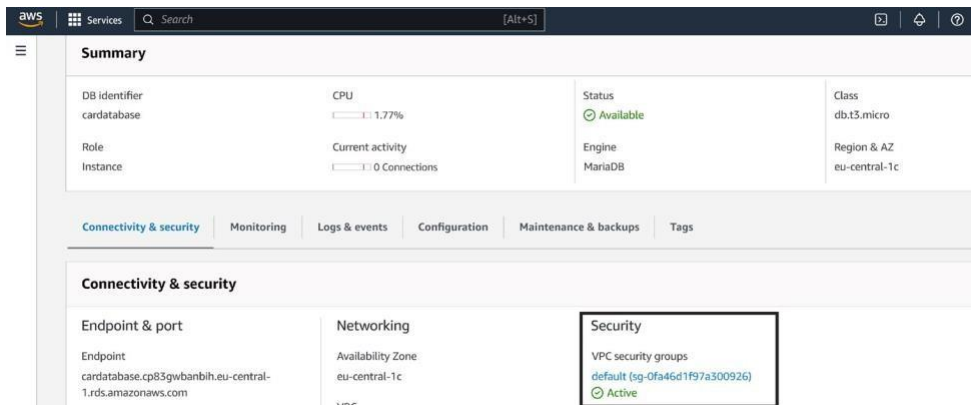


Figure 17.10: Connectivity & security

11. On the opening page, click the **Edit inbound rules** button from the **Inbound rules** tab. Click the **Add rule** button to add a new rule. For the new rule, select the **MySQL/Aurora** type and the **My IP** destination under the **Source** column. The **My IP** destination automatically adds the current IP address of your local computer as an allowed destination:

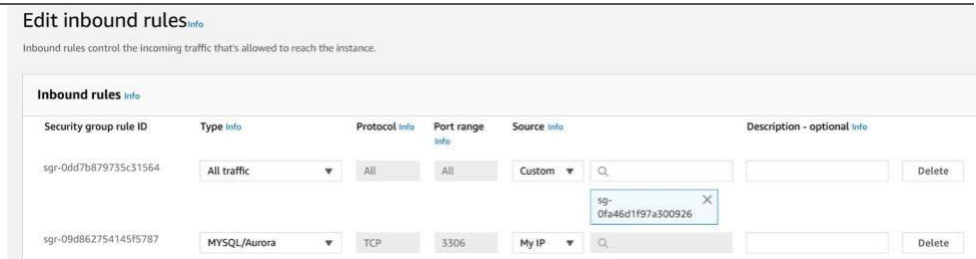


Figure 17.11: Inbound rules

12. After you have added a new rule, press the **Save rules** button.
13. Open the Spring Boot application that we created in *Chapter 5, Securing Your Backend*. Change the url, username, and password database settings in the application.properties file to match your Amazon RDS database. The format of the spring.datasource.url property value is jdbc:mariadb://your_rds_db_domain:3306/your_db_name, as shown in the following screenshot:

```

1 spring.jpa.show-sql=true
2 spring.datasource.url=jdbc:mariadb://cardatabase.cp83gwbanblh.eu-central-1.rds.amazonaws.com:3306/cardb
3 spring.datasource.username=admin
4 spring.datasource.password=q{4{4Xs]RI+RZhZIbs0}+!y6b;po77s
5 spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
6 spring.jpa.generate-ddl=true
7 spring.jpa.hibernate.ddl-auto=create-drop
8 spring.data.rest.basePath=/api

```

Figure 17.12: The application.properties file

14. Now, if you run your application, you can see from the console that database tables are created and demo data is inserted into our Amazon RDS database:

```

Console x JUnit
CardatabaseApplication [Java Application] C:\Program Files\Java\jdk-17.0.2\bin\javaw.exe (16.8.2023 klo 12.21.39) [pid: 9644]
2023-08-16T12:21:46.584+03:00 INFO 9644 --- [ restartedMain] c.p.cardatabase.CardatabaseApplication : Ford Mustang
2023-08-16T12:21:46.584+03:00 INFO 9644 --- [ restartedMain] c.p.cardatabase.CardatabaseApplication : Nissan Leaf
2023-08-16T12:21:46.585+03:00 INFO 9644 --- [ restartedMain] c.p.cardatabase.CardatabaseApplication : Toyota Prius
Hibernate: select next value for app_user_seq
Hibernate: insert into app_user (password,role,username,id) values (?,?,,?)
Hibernate: select next value for app_user_seq
Hibernate: insert into app_user (password,role,username,id) values (?,?,,?)

```

Figure 17.13: Console

15. In this phase, you should build your Spring Boot application. Run a Gradle build task in Eclipse by right-clicking **Project** in the Project Explorer, navigating to **Window | Show View | Other**, and selecting **Gradle | Gradle Tasks** from the list. This opens a list of Gradle tasks, and you can start the build process by double-clicking the **build** task. It will create a new JAR file in the build/libs folder.

We now have proper database settings, and we can use our newly built application when we deploy our application to AWS.

Deploying our Spring Boot application

After we have deployed our database to Amazon RDS, we can start to deploy our Spring Boot application. The Amazon service that we are using is **Elastic Beanstalk**, which can be used to run and manage web apps in AWS. There are other alternatives, such as AWS Amplify, that can be used as well. Elastic Beanstalk is available for the Free Tier, and it also supports a wide range of programming languages (for example, Java, Python, Node.js, and PHP).

The following steps will take you through the process of deploying our Spring Boot application to Elastic Beanstalk:

1. First, we have to create a new **role** for our application deployment. The role is needed to allow Elastic Beanstalk to create and manage your environment. You can create a role using the Amazon **IAM (Identity and Access Management)** service. Use the AWS search bar to navigate to the IAM service. In the IAM service, select **Roles** and press the **Create role** button. Select **AWS Service** and **EC2**, as shown in the following screenshot, and press the **Next** button:

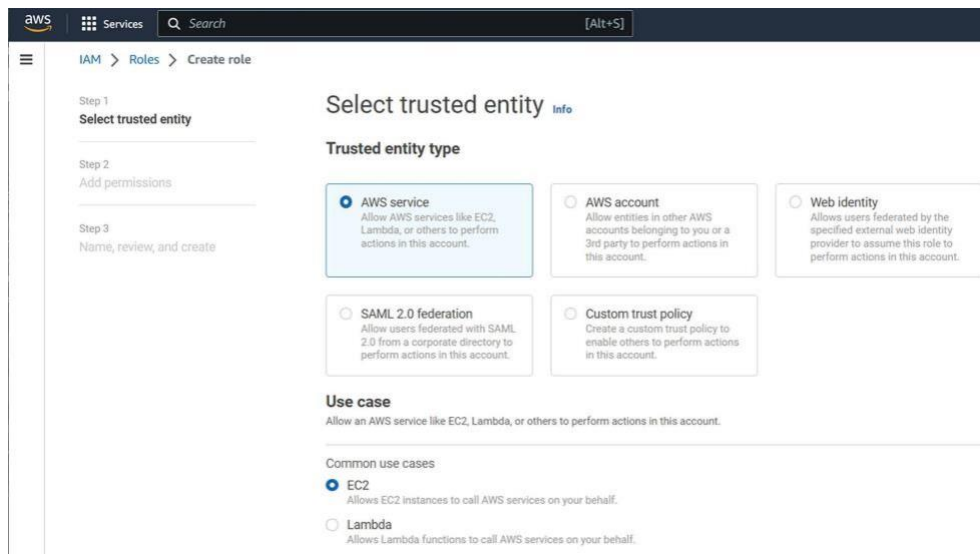


Figure 17.14: Create role

2. In the **Add Permissions** step, select the following permission policies: **AWSElasticBeanstalkWorkerTier**, **AWSElasticBeanstalkWebTier**, and

AWS Elastic Beanstalk Multicontainer Docker, then press the **Next** button. You can use the search bar to find the correct policies:

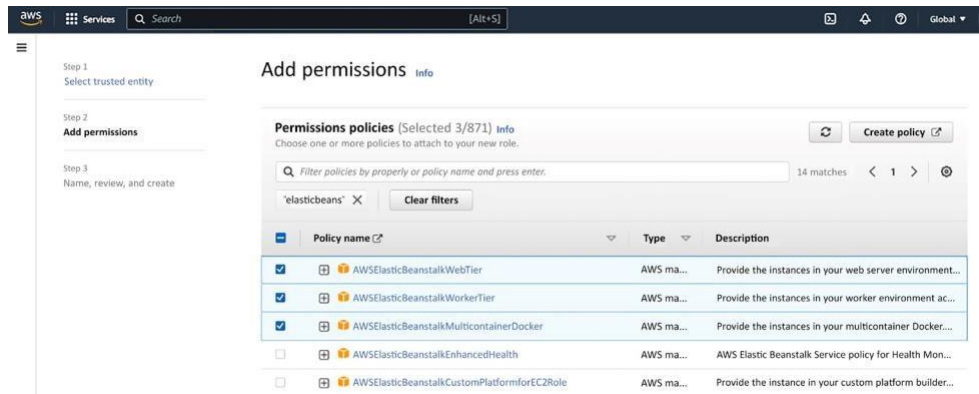


Figure 17.15: Add permissions



You can read more about managing Elastic Beanstalk instance profiles and policies at <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/iam-instanceprofile.html>.

3. Type a name for your role, as illustrated in the next screenshot, and finally press the **Create role** button:

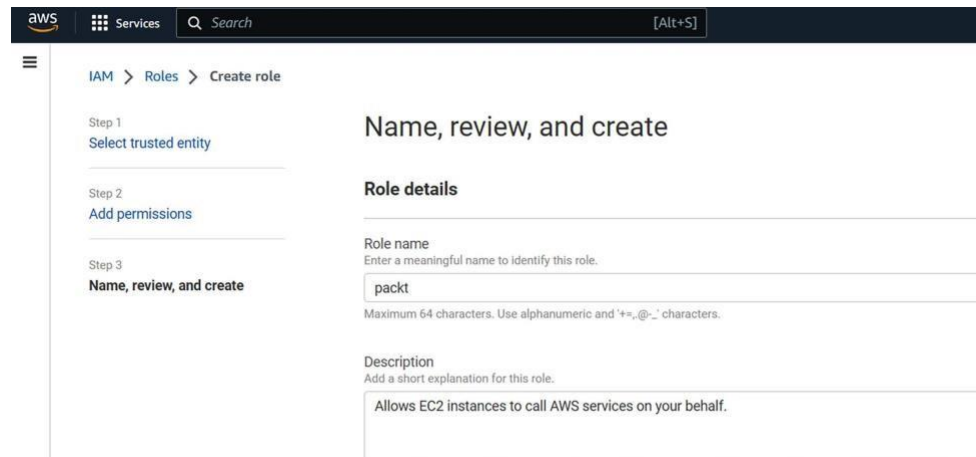


Figure 17.16: Role name

The new role that we just created allows Elastic Beanstalk to create and manage our environment. Now, we can start to deploy our Spring Boot application.

4. Use the AWS dashboard search bar to find the **Elastic Beanstalk** service. Click the service to navigate to the Elastic Beanstalk page:

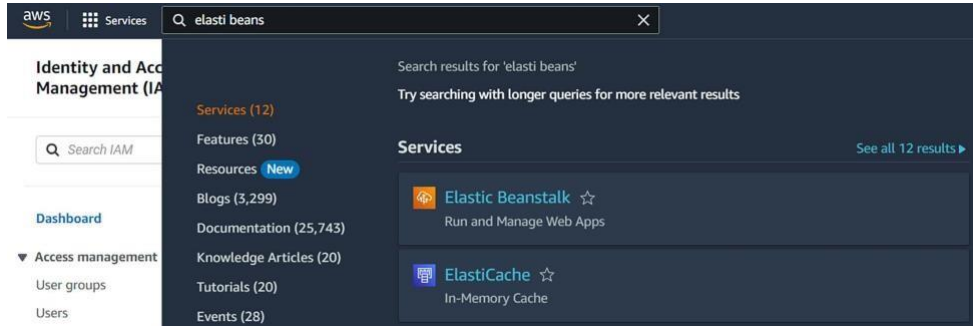


Figure 17.17: Elastic Beanstalk service

5. Click **Applications** in the left side menu, and press the **Create application** button to create a new application. Type a name for your application, as shown in the following screenshot, and press the **Create** button:

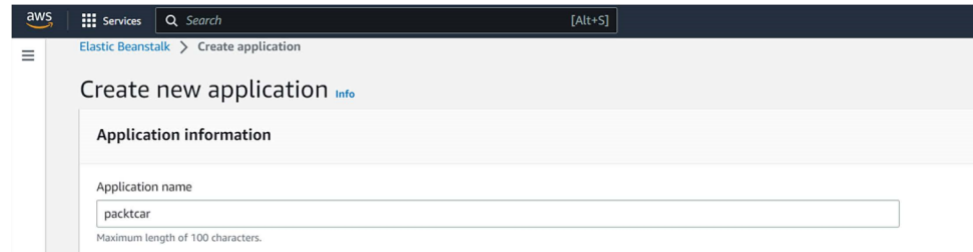


Figure 17.18: Create application

6. Next, we have to create an **environment** for our application. An environment is a collection of AWS resources running an application version. You can have multiple environments for one application: for example, development, production, and testing environments. Click the **Create new environment** button to configure a new environment:

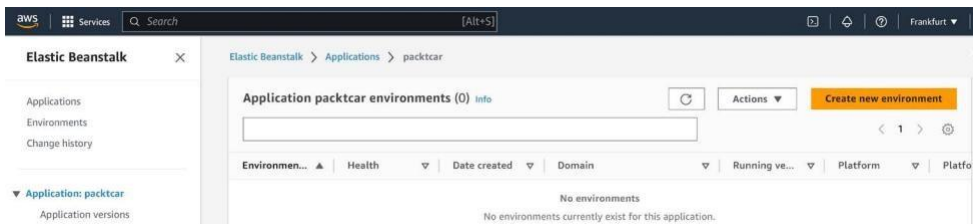
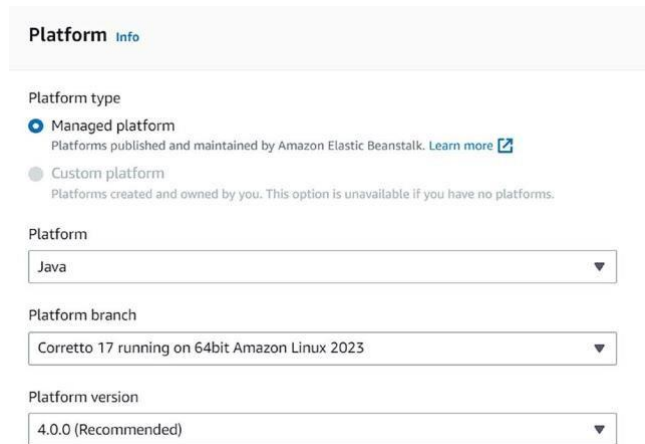


Figure 17.19: Create new environment

7. In the environment configuration, you first have to set the platform. In the **Platform type** section, select **Java** and the first version **17** for the branch, as shown in the

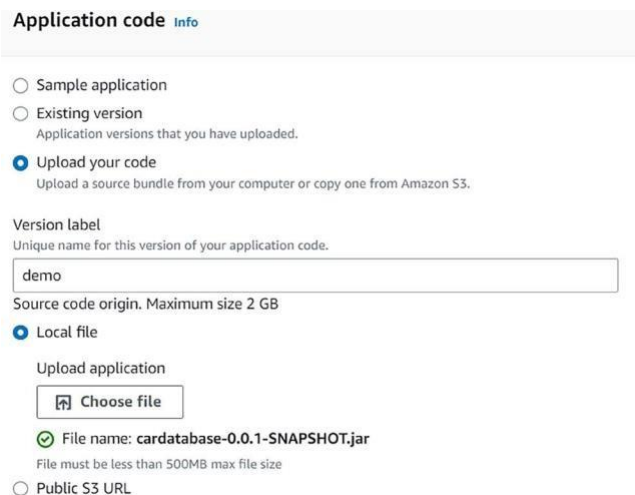
following screenshot. The **Platform version** is a combination of specific versions of an operating system, runtime, web server, application server, and Elastic Beanstalk components. You can use the recommended **Platform version**:



The screenshot shows the 'Platform' configuration section. It includes a header 'Platform' with an 'Info' link. Below it, the 'Platform type' is set to 'Managed platform' (selected with a radio button). Under 'Managed platform', there is a description: 'Platforms published and maintained by Amazon Elastic Beanstalk. [Learn more](#)'. The 'Custom platform' option is unselected. Below this, the 'Platform' dropdown is set to 'Java'. The 'Platform branch' dropdown is set to 'Corretto 17 running on 64bit Amazon Linux 2023'. The 'Platform version' dropdown is set to '4.0.0 (Recommended)'.

Figure 17.20: Platform type

- Next, go to the **Application code** section in the configuration page. Select **Upload your code** and **Local file**. Click the **Choose file** button and select the Spring Boot .jar file that we built earlier. You also have to type in a unique **Version label**. Finally, press the **Next** button:



The screenshot shows the 'Application code' configuration section. It includes a header 'Application code' with an 'Info' link. Below it, the 'Upload your code' option is selected (radio button). Under 'Upload your code', there is a description: 'Upload a source bundle from your computer or copy one from Amazon S3.'. The 'Existing version' option is unselected. Below this, the 'Version label' is set to 'demo'. The 'Source code origin' is set to 'Local file'. Under 'Local file', the 'Choose file' button is visible. Below the button, the file name is shown as 'cardatabase-0.0.1-SNAPSHOT.jar' with a green checkmark. Below the file name, there is a note: 'File must be less than 500MB max file size'. The 'Public S3 URL' option is unselected.

Figure 17.21: Create new environment

- In the **Configuration service access** step, select the role that you created earlier from the **EC2 instance profile** dropdown list, as shown in the following screenshot. Then, press the **Next** button:

Configure service access [Info](#)

Service access

IAM roles, assumed by Elastic Beanstalk as a service role, and EC2 instance profiles allow Elastic Beanstalk to create and manage your environment. Both the IAM role and instance profile must be attached to IAM managed policies that contain the required permissions. [Learn more](#)

Service role

☐ Create and use new service role

☒ Use an existing service role

Existing service roles

Choose an existing IAM role for Elastic Beanstalk to assume as a service role. The existing IAM role must have the required IAM managed policies.

aws-elasticbeanstalk-service-role

EC2 key pair

Select an EC2 key pair to securely log in to your EC2 instances. [Learn more](#)

Choose a key pair

EC2 instance profile

Choose an IAM instance profile with managed policies that allow your EC2 instances to perform required operations.

packt

[View permission details](#)

Figure 17.22: Service access

10. You can skip the optional **Set up networking, database, and tags** and **Configure instance traffic and scaling** steps.
11. Next, move to the **Configure updates, monitoring, and logging** step. In the **Environment properties** section, we have to add the following environment properties. You can add new properties by pressing the **Add environment property** button at the bottom of the page. There are already some predefined properties that you don't have to modify (GRADLE_HOME, M2 and M2_HOME):
 - SERVER_PORT: 5000 (Elastic beans have a Nginx reverse proxy that will forward incoming requests to internal port 5000).
 - SPRING_DATASOURCE_URL: The database URL you need to use here is identical to the database URL value we previously configured in the 'application.properties' file when we initially tested the AWS database integration.
 - SPRING_DATASOURCE_USERNAME: The username of your database.
 - SPRING_DATASOURCE_PASSWORD: The password of your database.

The following screenshot shows the new properties:

Environment properties

The following properties are passed in the application as environment properties. [Learn more](#)

Name	Value	
GRADLE_HOME	/usr/local/gradle	Remove
M2	/usr/local/apache-maven/bin	Remove
M2_HOME	/usr/local/apache-maven	Remove
SERVER_PORT	5000	Remove
SPRING_DATASOURCE_URL	jdbc:mariadb://cardatabase.cp83gwbanl	Remove
SPRING_DATASOURCE_USERNAME	admin	Remove
SPRING_DATASOURCE_PASSWORD	q{4{4%Xs}RI+RZFibsO+!y8b;p177s	Remove
Add environment property		

Figure 17.23: Environment properties

12. Finally, in the **Review** step, press the **Submit** button, and your deployment will start. You have to wait until your environment is successfully launched, as illustrated in the next screenshot. The **Domain** in the **Environment overview** is the URL of your deployed REST API:

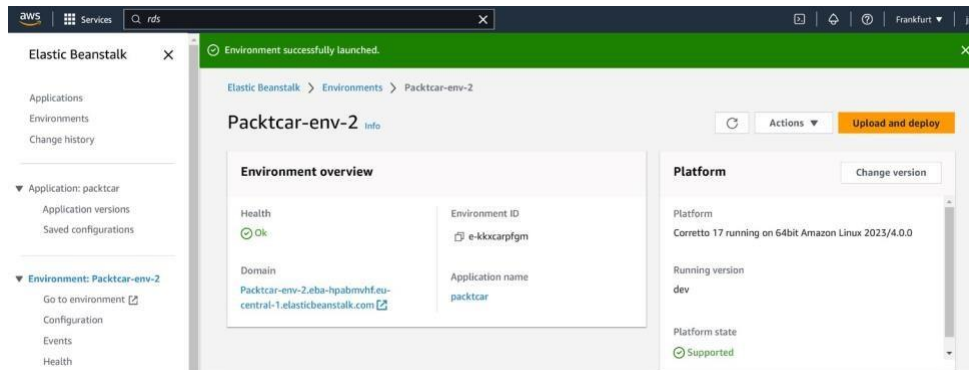


Figure 17.24: Environment successfully launched

13. Now, we have deployed our Spring Boot application, but the application can't access the AWS database yet. For this, we have to allow access from the deployed application to our database. To do this, navigate to Amazon RDS and select your database from the RDS database list. Then, click **VPC security groups** and click the **Edit inbound rules** button, like we did earlier. Delete the rule that allows access from your local IP address.

14. Add a new rule whose **Type** is **MySQL/Aurora**. In the **Destination** field, type in sg. This will open a list of environments, as shown in the following screenshot. Select the environment where your Spring Boot application is running (it begins with the “awseb” text and has a subtitle that shows the name of your environment) and press the **Save rules** button:

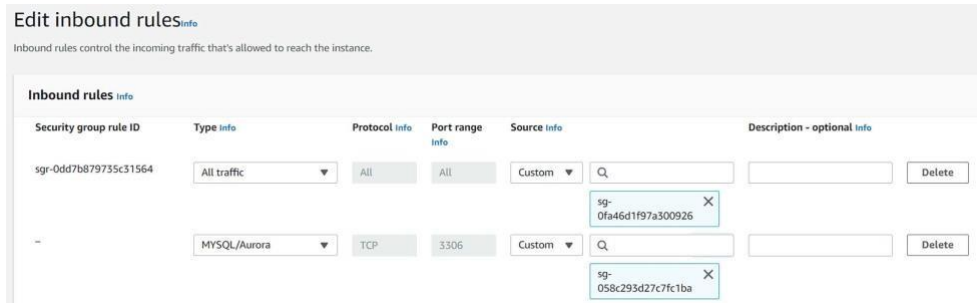


Figure 17.25: Inbound rules

15. Now, your application is properly deployed, and you can log in to your deployed REST API using Postman and the URL that you got from the domain in *step 12*. The following screenshot shows the POST request that is sent to the `aws_domain_url/login` endpoint:

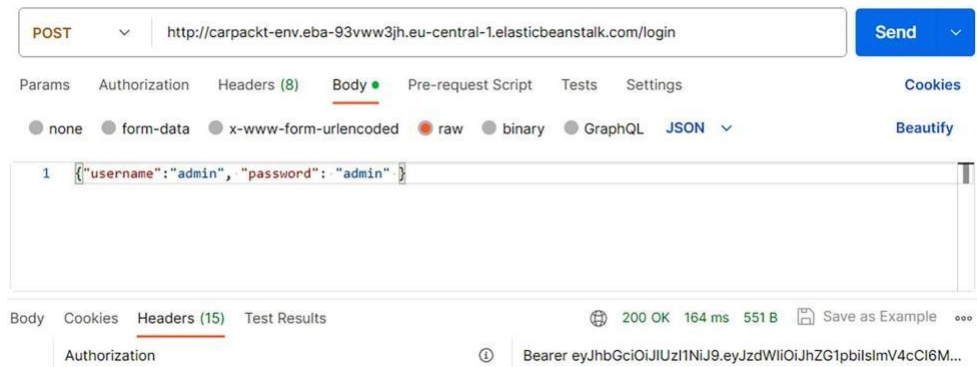


Figure 17.26: Postman authentication

You can also configure a custom domain name for your Elastic Beanstalk environment, and then you can use HTTPS to allow users to connect to your website securely. If you don’t own a domain name, you can still use HTTPS with a self-signed certificate for development and testing purposes. You can find the configuration instructions in the AWS documentation: <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/configuring-https.html>.



Note! You should delete AWS resources that you have created to avoid being charged unexpectedly. You will get a reminder email from AWS to delete resources before the end of your Free Tier period.

Now, we are ready to deploy our frontend.

Deploying the frontend with Netlify

Before we deploy with Netlify, we will learn how you can build your React project locally.

Move to your frontend project folder and execute the following `npm` command:

```
npm run build
```

By default, your project is built in the `/dist` folder. You can change the folder by using the `build.outDir` property in your Vite configuration file.

First, the build process compiles your TypeScript code; therefore, you have to fix all TypeScript errors or warnings, if there are any. A commonly encountered error occurs when you forget to remove unused imports, as illustrated in the example error below:

```
src/components/AddCar.tsx:10:1 - error TS6133: 'Snackbar' is declared but  
its value is never read. 10 import Snackbar from  
'@mui/material/Snackbar';
```

This indicates that the `AddCar.tsx` file imports the `Snackbar` component, but the component isn't actually utilized. Therefore, you should remove this unused import. Once all errors have been resolved, you can proceed to rebuilding your project.

Vite uses **Rollup** (<https://rollupjs.org/>) to bundle your code. Test files and development tools are *not* included in the production build. After you have built your app, you can test your local build using the following `npm` command:

```
npm run preview
```

The command starts a local static web server that serves your built app. You can test your app in a browser by using the URL that is shown in the terminal.

You could deploy your frontend to AWS as well, but we will use **Netlify** (<https://www.netlify.com/>) for our frontend deployment. Netlify is a modern web development platform that is easy to use. You can use the Netlify **command-line interface (CLI)** or GitHub to deploy your project. In this section, we will use Netlify's GitHub integration to deploy our frontend:

1. First, we have to change our REST API URL. Open your frontend project with VS Code and open the `.env` file in the editor. Change the `VITE_API_URL` variable to match your backend's URL, as follows, and save the changes:

```
VITE_API_URL=https://carpackt-env.eba-whufxac5.eu-central-2.elasticbeanstalk.com
```

2. Create a GitHub repository for your frontend project. Execute the following Git commands in your project folder using the command line. These Git commands create a new Git repository, make an initial commit, set up a remote repository on GitHub, and push the code to your remote repository:

```
git init
git add .
git commit -m "first commit"
git branch -M main
git remote add origin <YOUR_GITHUB_REPO_URL>
git push -u origin main
```

3. Sign up and log in to Netlify. We will use a free **Starter** account that has limited features. With this account, you can build one concurrent build for free, and there is some limitation in bandwidth.



You can read more about the Netlify free account features at <https://www.netlify.com/pricing/>.

4. Open the **Sites** from the left side menu and you should see the **Import an existing project** panel, as shown in the following screenshot:

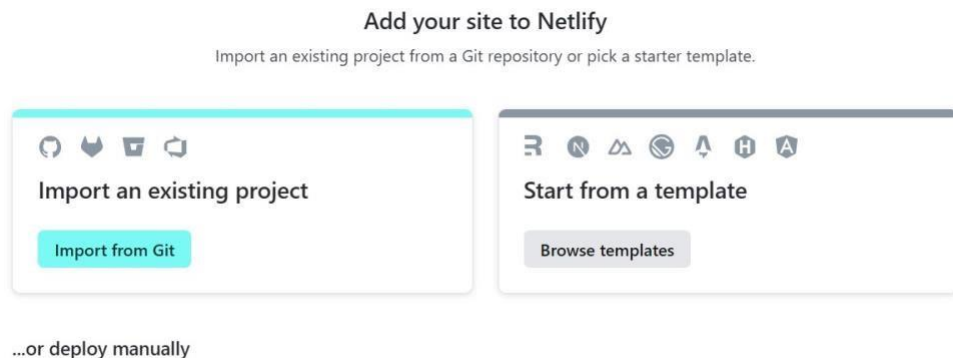


Figure 17.27: Import an existing project

5. Click the **Import from Git** button and select **Deploy with GitHub**. In this phase, you have to authorize your GitHub to get access to your repositories. After you have authorized successfully, you should see your GitHub username and repository search field, as shown in the following screenshot:

Let's deploy your project.



Figure 17.28: GitHub repository

6. Search for your frontend repository and click it.
7. Next, you will see the deployment settings. Continue with the default settings by pressing the **Deploy <your_repository_name>** button:

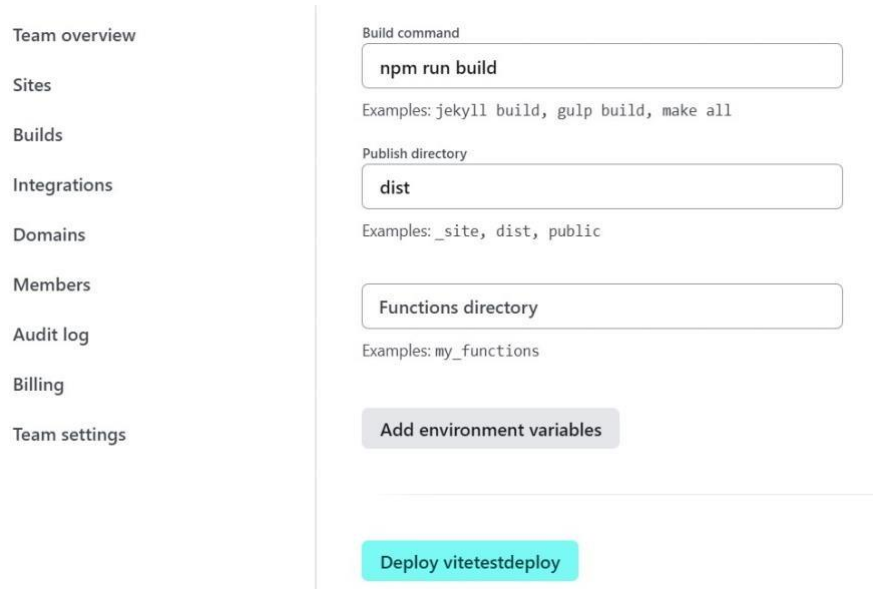


Figure 17.29: Deployment settings

8. After the deployment has finished, you will see the following dialog. Press the **View site deploy** button, as shown in the following figure, and you will be redirected to the **Deploys** page. Netlify generates a random site name for you, but you can use your own domain as well:

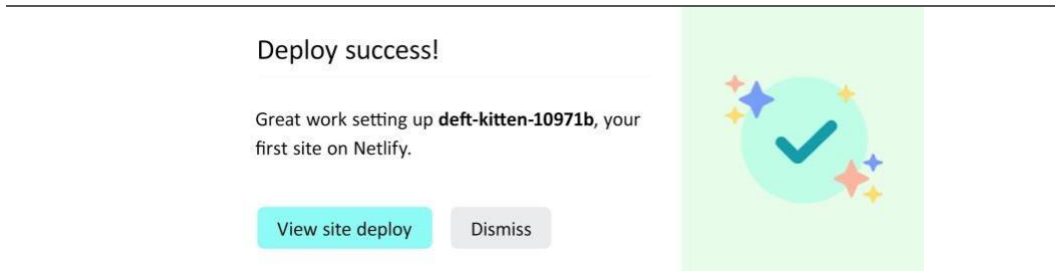


Figure 17.30: Deploy success

9. On the **Deploys** page, you will see your deployed site, and you can access your frontend by clicking the **Open production deploy** button:

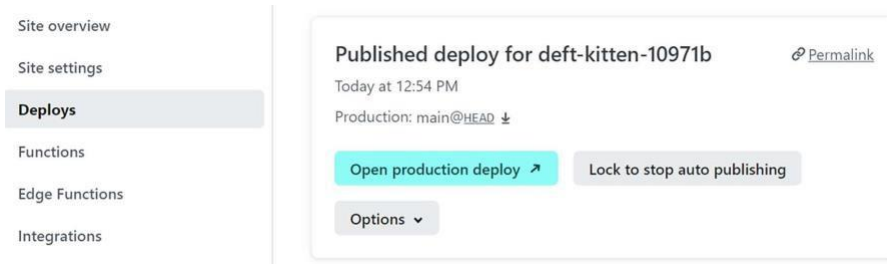


Figure 17.31: Deploys 10.

Now, you should see the login form, as follows:

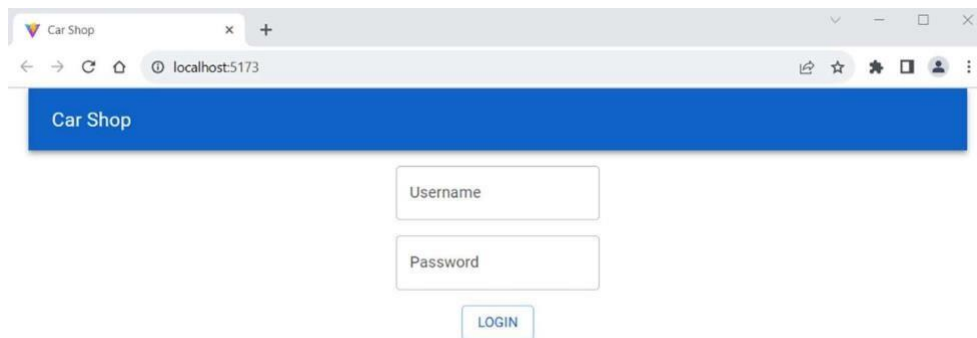


Figure 17.32: Login screen



We have now deployed our frontend, and we can move on to learning about containers.

Using Docker containers

Docker (<https://www.docker.com/>) is a container platform that makes software development, deployment, and shipping easier. Containers are lightweight and executable software packages that include everything that is needed to run software. Containers can be deployed to cloud services, such as AWS, Azure, and Netlify, and they offer many benefits for deploying applications:

- Containers are isolated, which means each container runs independently of the host system and other containers.
- Containers are portable because they contain everything an application needs to run.
- Containers can also be used to ensure consistency between development and production environments.



Note! To run Docker containers on Windows, you need the Windows 10 or 11 Professional or Enterprise versions. You read more about this in the Docker installation documentation:

<https://docs.docker.com/desktop/install/windows-install/>.

In this section, we will create a container for our MariaDB database and Spring Boot application, as follows:

1. Install Docker on your workstation. You can find installation packages for multiple platforms at <https://www.docker.com/get-docker>. If you have a Windows operating system, you can go through the installation wizard using the default settings.



If you are having problems with the installation, you can read the Docker troubleshooting documentation at <https://docs.docker.com/desktop/troubleshoot/topics>.

After the installation, you can check the current version by typing the following command in the terminal. Note! When you run Docker commands, you should start Docker Engine if it is not running (on Windows and macOS, you start Docker Desktop):

```
docker --version
```

2. First, we create a container for our MariaDB database. You can pull the latest MariaDB database image version from Docker Hub using the following command:

```
docker pull mariadb:latest
```

3. After the pull command has finished, you can check that a new mariadb image exists by typing the `docker image ls` command, and the output should look as follows. A **Docker image** is a template that contains instructions for creating a container:

```
PS C:\work\tmp\PacktTestCode\cardatabase_05c\build\libs> docker image ls
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
mariadb       latest    1a580bde192c   4 days ago    404MB
```

Figure 17.33: Docker images

4. Next, we will run the mariadb container. The `docker run` command creates and runs a container based on the given image. The following command sets the root user password and creates a new database, called `cardb`, that we need for our Spring Boot application (Note! Use your own MariaDB root user password that you are using in your Spring Boot application):

```
docker run --name cardb -e MYSQL_ROOT_PASSWORD=your_pwd -e MYSQL_
DATABASE=cardb mariadb
```

5. Now, we have created our database container, and we can start to create a container for the Spring Boot application. First, we have to change the data source URL of our Spring Boot application. Open the `application.properties` file of your application and change the `spring.datasource.url` value to the following:

```
spring.datasource.url=jdbc:mariadb://mariadb:3306/cardb
```

This is because our database is now running in the `cardb` container and the port is 3306.

6. Then, we have to create an executable JAR file from our Spring Boot application, just as we did at the beginning of this chapter. You can also run a Gradle task in the Eclipse by right-clicking **Project** in the Project Explorer, selecting **Window | Show View | Gradle** and then **Gradle Tasks** from the list. This opens a list of Gradle tasks, and you can start the build process by double clicking the **build** task. Once the build is finished, you can find the executable JAR file from the `build/libs` folder inside your project folder.
7. Containers are defined by using **Dockerfiles**. Create a new Dockerfile using Eclipse in the root folder of your project (`cardatabase`) and name it `Dockerfile`. The following lines of code show the contents of the Dockerfile:

```
FROM eclipse-temurin:17-jdk-alpine
VOLUME /tmp
EXPOSE 8080
COPY build/libs/cardatabase-0.0.1-SNAPSHOT.jar app.jar ENTRYPOINT
["java", "-jar", "/app.jar"]
```

Let's examine each line:

- FROM defines the **Java Development Kit (JDK)** version, and you should use the same version that you used to build your JAR file. We are using Eclipse Temurin, which is an open-source JDK, and version 17, which we used when we developed our Spring Boot application.
- Volumes are used for persistent data generated by and used by Docker containers.
- EXPOSE defines the port that should be published outside of the container.
- COPY copies the JAR file to the container's filesystem and renames it app.jar.
- Lastly, ENTRYPOINT defines the command-line arguments that the Docker container runs.



You can read more about the Dockerfile syntax at <https://docs.docker.com/engine/reference/builder/>.

8. Build an image with the following command in the folder where your Dockerfile is located.

With the `-t` argument, we can give a friendly name to our container:

```
docker build -t carbackend .
```

9. At the end of the build, you should see a **Building [...] FINISHED** message, as illustrated in the following screenshot:

```
PS C:\work\cardatabase> docker build -t carbackend .
[+] Building 1.4s (7/7) FINISHED
=> [internal] load build definition from Dockerfile                                0.0s
=> => transferring dockerfile: 32B                                                0.0s
=> [internal] load .dockerignore                                                  0.0s
=> => transferring context: 2B                                                    0.0s
=> [internal] load metadata for docker.io/adoptopenjdk/openjdk11:latest         1.3s
=> [internal] load build context                                                 0.0s
=> => transferring context: 87B                                                  0.0s
=> [1/2] FROM docker.io/adoptopenjdk/openjdk11:latest@sha256:ec7b97fa188861542ae8af31fbdad43f2f133170b43d29204ad 0.0s
=> CACHED [2/2] COPY target/cardatabase-0.0.1-SNAPSHOT.jar app.jar              0.0s
=> exporting to image                                                            0.0s
=> => exporting layers                                                            0.0s
=> writing image sha256:cc38e3bab5abdbd8c97f8d5e1012a444263ffcb98d75ef8682efedf277de535c 0.0s
=> => naming to docker.io/library/carbackend                                    0.0s
```

Figure 17.34: Docker build

10. Check the list of images using the `docker image ls` command. You should see two images now, as shown in the following screenshot:

```
PS C:\work\tmp\PacktTestCode\cardatabase_05c> docker image ls
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
carbackend     latest    f550cb07c922   5 minutes ago 356MB
mariadb        latest    1a580bde192c   4 days ago    404MB
```

Figure 17.35: Docker images

11. Now, we can run our Spring Boot container and link the MariaDB container to it using the following command. This command specifies that our Spring Boot container can access the MariaDB container using the mariadb name:

```
docker run -p 8080:8080 --name carapp --link cardb:mariadb -d
carbackend
```

12. When our application and database are running, we can access the Spring Boot application logs using the following command:

```
docker logs carapp
```

We can see here that our application is up and running:

```
Hibernate: select next value for owner_seq
Hibernate: select next value for owner_seq
Hibernate: insert into owner (firstname,lastname,ownerid) values (?,?,?)
Hibernate: insert into owner (firstname,lastname,ownerid) values (?,?,?)
Hibernate: select next value for car_seq
Hibernate: select next value for car_seq
Hibernate: insert into car (brand,color,model,model_year,owner,price,register_number,id) values (?,?,?, ?, ?, ?, ?, ?)
Hibernate: insert into car (brand,color,model,model_year,owner,price,register_number,id) values (?,?,?, ?, ?, ?, ?, ?)
Hibernate: insert into car (brand,color,model,model_year,owner,price,register_number,id) values (?,?,?, ?, ?, ?, ?, ?)
Hibernate: select c1_0.id,c1_0.brand,c1_0.color,c1_0.model,c1_0.model_year,c1_0.owner,c1_0.price,c1_0.register_number fr
om car c1_0
2023-08-03T10:43:41.284Z INFO 1 --- [           main] c.p.cardatabase.CardatabaseApplication : Ford Mustang
2023-08-03T10:43:41.284Z INFO 1 --- [           main] c.p.cardatabase.CardatabaseApplication : Nissan Leaf
2023-08-03T10:43:41.284Z INFO 1 --- [           main] c.p.cardatabase.CardatabaseApplication : Toyota Prius
Hibernate: insert into app_user (password,role,username) values (?,?,?)
Hibernate: insert into app_user (password,role,username) values (?,?,?)
```

Figure 17.36: Application log

Our application has started successfully, and the demonstration data has been inserted into the database that exists in the MariaDB container. Now, you can use your backend, as seen in the following screenshot:

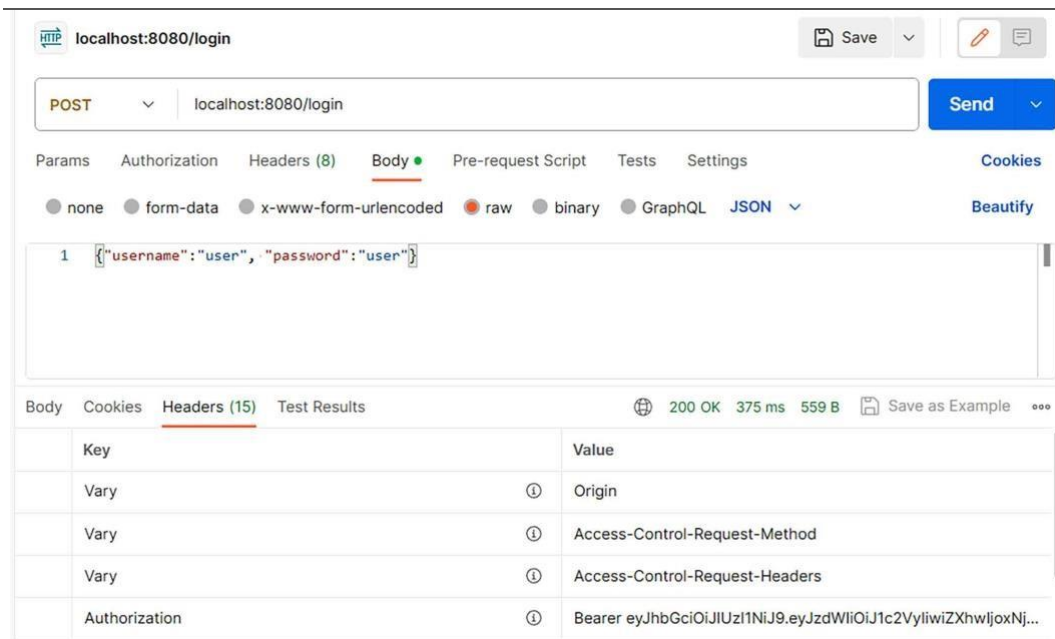


Figure 17.37: Application login

We have learned a few different ways to deploy your full-stack application and how to containerize your Spring Boot application. As a next step, you could study how to deploy Docker containers.

For example, AWS has a guide to deploying containers on Amazon ECS:

<https://aws.amazon.com/getting-started/hands-on/deploy-docker-containers/>.